

ResearchOS (KOI):
агент как штатный исследователь
Опыт разработки платформы с Claude / Cursor / Codex

...

...

15 августа 2026 г.

ПРОБЛЕМА → гипотезы → методы → канбан
→ отчёт → вердикт + инсайты → база знаний

Ключевой принцип

Знание рождается только из эксперимента с **заранее заданным правилом решения**.
Вердикт — подстановка чисел, не «впечатление» модели.

- Хранение: Markdown-файлы, без БД
- UI: локальный редактор `project.md` / отчётов / канбана
- Агент IDE — исполнитель по умолчанию

Можно портировать скиллы между Claude / Cursor / Codex

SKILL.md — универсальный формат плейбука;

AGENTS.md — единая точка входа для любого агентного IDE.

- Общие плейбуки живут в `agents/skills/` и не привязаны к Cursor
- Агент адаптирует пути и команды под свою среду
- Один репозиторий — одинаковые workflow для всей команды

Git + агент вместо общего хоста

У каждого — свой локальный инстанс (localhost:8080),
данные проектов синхронизируются через git по инициативе агента.

- Синхронизируется только `projects/<id>/`
- Скилл `koi-project-sync`: pull при старте, push значимых изменений
- Хуки `sessionStart` / `stop` — без отдельного CI

Инсайт 3: веб-платформа без API агента

Hooks + inbox + файловая очередь

UI → agent-chat-queue.json → watcher → агент → ответ в UI

Режим	Задержка	API	Один чат	Inbox: JSON-очередь →
Inbox-чат	~1–3 с	нет	да	
Hooks	до минут	нет	нет	
Фоновый API	~5 с	да	нет	

koi_agent_chat_inbox.py pending → скиллы koi-agent-chat / koi-related-work

Инсайт 4: разделение интеллекта и транспорта

Агент делает:

- эксперимент
- анализ и синтез
- курирование знаний
- ответы в чате UI

Код делает:

- валидация формата отчёта
- idempotent ingest
- пересборка KNOWLEDGE.md
- очереди и хуки

Форматные ворота

Невалидный `.run.md` отбраковывается целиком — состояние не меняется.

Инсайт 5: двухслойная база знаний

Автоген (правила)

KNOWLEDGE.md, hypotheses.md, KNOWLEDGE_LOG.md

Точно, плоско, пересобирается при каждом save

Курируемый слой (агент)

projects/<id>/knowledge/*.md

Кросс-анализ: связки, противоречия, пределы применимости, открытые вопросы

Агент в чате сначала читает research.json, отчёты — только если не хватает деталей.

Инсайт 6: файловые очереди как оркестратор

- `agent-chat-queue.json` — вопросы из UI
- `done-research queue` — карточки, ждущие вывода
- `sync-push queue` — что коммитить в git

При старте сессии агент:

- 1 pull (синхронизация)
- 2 проверка всех очередей
- 3 (опционально) старт dev-сервера

Нет центрального планировщика — только файлы + хуки + дисциплина агента.

Инсайт 7: агент как QA UI

- Скилл `koi-ui-design-review`: линт, design tokens, a11y
- Скилл `koi-visual-qa`: скриншоты, регресс mindmap/канбана/модалок
- Агент проверяет UI, который сам же и пишет

Клон без облачного бэкенда — нормальный случай

`koi_check_hypothesis.py` недоступен $\Rightarrow$ агент сам:

- 1 проводит эксперимент локально
- 2 заполняет шаблон `experiment-report.md`
- 3 `-ingest-only` (сначала `-dry-run`)

Репозиторий спроектирован так, чтобы агент **IDE** заменял серверный воркер.

- 1 MD + git + очереди — минимальный стек интеграции агентов
- 2 Скиллы — переносимые плейбуки, не привязка к одному IDE
- 3 Форматные ворота — LLM не пишет в БЗ напрямую
- 4 Агент = исследователь + синхронизатор + QA, код = транспорт
- 5 Локальный хост + git — командная работа без общего сервера

Демо (если есть время)

- Карта гипотез + канбан на localhost:8080
- Панель «Спросить агента» → Inbox-чат
- База знаний: вердикты, инсайты, журнал пополнений